PYTHON SUPERHUMAN MASTERY ROADMAP

Objective: From Zero to FAANG-Level Python + Data Science Expert

Total Duration: 8 Weeks | Daily Commitment: 8 Hours | Intensity: Maximum

---

📊 ROADMAP STRUCTURE

PHASE 1: PYTHON FOUNDATIONS (WEEKS 1-2)

Goal: Understand Python at machine level, not surface level

Week 1: Python Internals & Execution Model

Day 1-2: Python Execution Deep Dive

- How Python code executes (lexical analysis → parsing → AST → bytecode → PVM)
- Interpreter vs Compiler
- CPython internals (Python written in C)
- Bytecode disassembly (`dis` module)
- Source → Object files → Execution pipeline
- GIL (Global Interpreter Lock) conceptually

Day 3-4: Variables, Objects & Memory Allocation

- Everything is an object in Python
- Object structure: id, type, value
- Reference vs Value semantics
- Memory addresses and `id()` function
- Mutable vs Immutable (deep)
- Reference counting & Garbage Collection
- Memory fragmentation & optimization

Day 5-6: Data Types - The Foundation

- **Numeric**: int, float, complex (internal representation)

- **Sequence**: str, bytes, list, tuple (how stored in memory)
- **Collections**: dict, set, frozenset (hash tables, collision handling)
- **Special**: bool, NoneType
- Type casting & coercion rules
- Dynamic typing implications

Day 7: Operators & Expressions

- Arithmetic, comparison, logical, bitwise, assignment operators
- Operator precedence & associativity (detailed)
- Short-circuit evaluation
- Augmented assignment (`+=`, etc.) internals
- Expression evaluation order

Week 2: Control Flow & Functions (Foundations)

Day 8-9: Control Flow

- if/elif/else (branch prediction in CPUs)
- for loops (iteration protocol)
- while loops (termination analysis)
- break, continue, pass
- Nested structures
- Performance implications

Day 10-11: Functions - The Backbone

- Function definition & call mechanism
- Parameters vs Arguments
- Default parameters (mutable default pitfall!)
- *args and **kwargs (unpacking internals)
- Return values & None
- Scope: LEGB rule (Local, Enclosing, Global, Built-in)
- Closures & nested functions

- Lambda functions (when & why)

Day 12-13: Advanced Function Concepts

- First-class functions (functions as objects)
- Higher-order functions
- Decorators (concept introduction)
- Recursion (stack overflow, tail recursion)
- Generator functions & yield
- Function introspection (`inspect` module)

Day 14: Week 2 Consolidation

- Practice: 20+ problems across all concepts
- Interview cross-questioning
- Identify weak areas

PHASE 2: ADVANCED PYTHON (WEEKS 3-4)

Goal: Mastery of Pythonic patterns, optimization, and design

Week 3: OOP & Design Patterns

Day 15-16: Classes & Objects

- Class definition & instantiation
- `__init__` constructor
- Instance vs Class attributes
- Methods (instance, class, static)
- `self` parameter (what it really is)
- Dunder methods (`__str__`, `__repr__`, `__len__`, etc.)
- Object lifecycle

Day 17-18: Inheritance & Polymorphism

- Single, multiple, multilevel inheritance

- Method resolution order (MRO)
- `super()` function
- Method overriding & overloading
- Abstract base classes (ABC)
- Composition vs Inheritance

Day 19-20: Advanced OOP

- Property decorators (@property)
- Descriptors (deep dive)
- Metaclasses (conceptual)
- Mixins
- Design patterns (Singleton, Factory, Observer)

Day 21: Advanced Functions & Decorators

- Decorator anatomy (wrapper functions)
- Built-in decorators (@staticmethod, @classmethod, @property)
- Custom decorators
- Decorator chaining
- Functools (partial, wraps, lru_cache)

Week 4: Iterators, Generators & Functional Programming

Day 22-23: Iterators & Generators

- Iterator protocol (`__iter__`, `__next__`)
- Generator functions (yield)
- Generator expressions
- yield from
- Lazy evaluation (memory efficiency)

Day 24-25: Functional Programming

- map, filter, reduce

- list/dict/set comprehensions (deep)
- Lambda functions (practical use)
- Higher-order functions
- Functools module (partial, reduce, wraps)

Day 26-27: Exception Handling

- Try-except-else-finally
- Raising exceptions
- Custom exceptions
- Exception hierarchy
- Context managers (`with` statement)
- `__enter__` and `__exit__` protocol

Day 28: Week 4 Consolidation

- 30+ intermediate problems
- Cross-questioning on OOP & functional concepts

PHASE 3: DATA STRUCTURES & ALGORITHMS (WEEKS 5-6)

Goal: Competitive programming level proficiency

Week 5: Data Structures in Python

Day 29-30: Lists, Tuples, Strings

- List operations (append, extend, insert, remove, pop)
- List slicing & negative indexing
- String manipulation (methods, formatting)
- Tuple unpacking
- Time complexity of operations

Day 31-32: Dictionaries & Sets

- Dict operations (get, keys, values, items)

- Dict comprehensions
- Set operations (union, intersection, difference)
- Hash functions & collisions
- Time complexity analysis

Day 33-34: Custom Data Structures

- Linked Lists (singly, doubly)
- Stacks & Queues
- Trees (binary, BST)
- Graphs (adjacency matrix, adjacency list)
- Heaps & priority queues

Day 35: Algorithms - Searching & Sorting

- Linear search, binary search
- Bubble, insertion, merge, quick, heap sort
- Time/space complexity (Big O)
- Practical optimization

Week 6: Advanced Algorithms & Problem-Solving

Day 36-37: Recursion & Backtracking

- Tail recursion vs regular recursion
- Backtracking problems (N-queens, subset)
- Memoization & dynamic programming intro

Day 38-39: Dynamic Programming

- Memoization vs Tabulation
- Classic DP problems (Fibonacci, knapsack, LCS)
- State representation

Day 40-41: Graph Algorithms

- DFS, BFS

- Dijkstra's algorithm
- Floyd-Warshall
- Topological sort

Day 42: Week 6 Consolidation

- 50+ algorithm problems (LeetCode medium/hard level)
- Algorithm pattern recognition

PHASE 4: NUMERICAL PYTHON & DATA SCIENCE (WEEKS 7-8)

Goal: Industry-ready data science fundamentals

Week 7: NumPy & Pandas Foundations

Day 43-44: NumPy Deep Dive

- ndarray structure (dtype, shape, strides)
- Broadcasting (rules & internals)
- Vectorization vs loops (performance)
- Memory layout (C-order vs F-order)
- Fancy indexing
- Universal functions (ufuncs)

Day 45-46: Pandas Essentials

- Series & DataFrame structure
- Indexing & selection
- Data cleaning & preprocessing
- Groupby & aggregation
- Merging & joining
- Time series handling

Day 47-48: Exploratory Data Analysis (EDA)

- Data profiling

- Visualization (matplotlib, seaborn basics)
- Statistical summaries
- Correlation & covariance
- Missing data handling

Day 49: Advanced Data Manipulation

- Pivot tables
- Window functions
- Categorical data
- Performance optimization in pandas

Week 8: Statistics, ML Fundamentals & Next Steps

Day 50-51: Statistics for Data Science

- Descriptive statistics
- Probability distributions
- Hypothesis testing (conceptual)
- Correlation vs causation
- Sampling & bias

Day 52-53: Machine Learning Fundamentals

- Supervised vs unsupervised learning
- Train/test split
- Overfitting/underfitting
- Linear regression from scratch
- Scikit-learn introduction

Day 54-55: ML Model Evaluation

- Metrics (accuracy, precision, recall, F1)
- Cross-validation
- Hyperparameter tuning

- Model selection

Day 56: Capstone & Final Assessment

- End-to-end ML project
- Full assessment of learning
- Identification of specialization path

🎯 LEARNING METHODOLOGY

For Each Concept:

1. **Understand WHY** (theory, internal workings)
2. **Learn HOW** (implementation, patterns)
3. **See WHAT** (code examples, visualization)
4. **Practice WHERE** (hands-on problems)
5. **Apply WHEN** (real-world scenarios)

After Each Day:

- ✅ Code written (not just read)
- ✅ Concept explained back to me
- ✅ 5-10 practice problems solved
- ✅ Common pitfalls identified

After Each Week:

- ✅ 20+ comprehensive problems
- ✅ Interview-style cross-questioning
- ✅ Learning pattern analysis
- ✅ Roadmap adjustment (if needed)

After Each Phase:

- ✅ Phase assessment (projects)
- ✅ Weak area identification

- ✅ Targeted reinforcement

📈 ASSESSMENT CRITERIA

Each Concept Will Be Tested On:

1. **Conceptual Understanding -** Can you explain internals?
2. **Implementation -** Can you code it from scratch?
3. **Optimization -** Can you optimize it?
4. **Application -** Can you use it in real problems?
5. **Edge Cases -** Can you handle gotchas?

🏁 SUCCESS METRICS

By End of Week 8:

- ✅ 500+ Python problems solved
- ✅ 10+ projects completed
- ✅ Deep understanding of Python internals
- ✅ FAANG interview-ready
- ✅ ML fundamentals mastered
- ✅ Code optimization skills
- ✅ System design thinking

📝 IMPORTANT NOTES

- **No memorization -** Understanding only
- **No skipping -** Every concept fully covered
- **Daily assessment -** Progress tracking
- **Adaptive learning -** Roadmap adjusts based on YOUR pace
- **Superhuman target -** Not just competent, but exceptional

---

🚀 STARTING POINT

We begin with: Python Execution Model Deep Dive (Day 1-2)

This is the FOUNDATION. Master this, and everything else clicks into place.

Ready? Let's make you superhuman. 💪